A Unified Projection Framework for Tensor-Structured Dictionary Learning, with an Efficiency Ranking System

Sahaj Chhetri

July 27, 2026

Abstract

Dictionary learning represents signals as sparse combinations of learned atoms which is mean to reduce the memory, but for multiway data (images, video, hyperspectral cubes) the standard approach vectorizes each patch, discarding spatial structure and inflating the dictionary to $\prod_i n_i \times K$ free parameters. Tensor decompositions repair this by constraining the *atoms* to a low-parameter format. We place five tensor-structured dictionaries—Canonical Polyadic (CP), Tucker, Tensor-Train (TT), Tensor-Ring (TR) and Block-Term Decomposition (BTD)—together with an unstructured baseline: identical Orthogonal Matching Pursuit (OMP) sparse coding and identical reconstruction operate on a shared *effective dictionary*, so that the only thing that varies between methods is how each atom is parameterized and how many parameters that costs. Four of the six methods differ only by a projection ("snap") step inside the K-SVD atom update; Tucker is realized as a Kronecker dictionary with a separable alternating-least-squares update. We benchmark on synthetic mixed-structure patches, real RGB image patches, and a hyperspectral cube, and we build a three-layer efficiency ranking system (Pareto dominance, an interpretable accuracy-per-parameter scalar, and profile-weighted composite tiers). Our central finding is that the "best" decomposition is strongly regime-dependent: the unstructured baseline wins on rich clean synthetic data; structured dictionaries win on noisy real patches, where structure acts as a regularizer; and at *matched parameter budget* on the hyperspectral cube the ranking neither favours Tensor-Train despite a large third mode, nor flips with noise. We also show that a fairness correction to the Tucker optimizer moves it from last place to a Pareto-dominant winner ($30.7 \to 32.2$ dB at identical size), a concrete demonstration that implementation confounds can dominate structural differences. We report all numbers, release the harness, and are explicit about the limitations of comparing projection variants rather than canonical published algorithms.

1 Introduction

Classical dictionary learning (DL) models a signal $y \in \mathbb{R}^n$ as $y \approx Dx$, where $D \in \mathbb{R}^{n \times K}$ is an overcomplete dictionary whose columns are unit-norm *atoms* and x is sparse. K-SVD [1] learns D by alternating sparse coding with a rank-one singular-value update of each atom. For multiway signals—an $n_1 \times n_2 \times n_3$ image or hyperspectral patch—the standard recipe flattens each patch to a vector of length $n = \prod_i n_i$. This vectorization has two costs. First, it discards the multiway structure (spatial neighbourhoods, spectral smoothness) that makes the data compressible. Second, the dictionary carries $n \cdot K$ free numbers, which grows multiplicatively with tensor order and invites over-fitting.

Tensor decompositions [3] address both problems by constraining atoms (or the dictionary as a whole) to a low-parameter tensor format. The literature contains many such constructions—separable/Kronecker dictionaries [8, 9], CP-structured atoms, and tensor-network dictionaries—but they are usually studied in isolation, each with its own optimizer, which makes cross-method

comparison unreliable: an observed gap may reflect the optimizer or the hand-picked settings rather than the decomposition. Our goal is a *controlled* comparison.

Contributions.

1. A unified harness in which CP, Tucker, TT, TR and BTD dictionaries, plus an unstructured baseline, share the same OMP sparse coding, the same reconstruction, and the same evaluation, so only the atom structure differs (Section 3).
2. A per-atom *projection* view that realizes CP/TT/TR/BTD as one line changed inside K-SVD, and a matched in-harness Tucker via separable alternating least squares (Section 4).
3. A three-layer efficiency ranking system—objective Pareto dominance, an interpretable accuracy-per-parameter scalar, and profile-weighted tiers (Section 6).
4. Experiments on synthetic, real RGB and hyperspectral data, including a matched-parameter-budget rate–distortion study across noise levels, with an explicit account of when and why the ranking changes (Sections 5–8).

2 Background: Dictionary Learning

2.1 The projection view of the K-SVD atom update

Given the dataset's signals, K-SVD learns a dictionary whose atoms sparsely represent them by alternating two steps. The dictionary D is initialized before the loop begins which is randomly initialized, or a fixed analytic dictionary.Then in the sparse-coding step, orthogonal matching pursuit (§2.2) holds the dictionary fixed and decides which atoms each signal uses and with what coefficients, producing the codes X. In the dictionary-update step (§2.3), the atoms are refined one at a time while the sparsity pattern of X is held fixed: for each atom, the support stays as OMP chose it, but the atom and its nonzero coefficients are updated together from the rank-one approximation of the error that atom is responsible for. The two steps alternate until the reconstruction error stops decreasing, which drives the dictionary toward a local optimum.

2.2 Orthogonal matching pursuit

Orthogonal matching pursuit (OMP) is the sparse-coding step of the loop, and it is what runs first in each K-SVD iteration to produce the codes X that the dictionary-update step of §2.3 then holds fixed. For a single signal y and dictionary $D = [d_1, \ldots, d_M]$, OMP selects atoms one at a time and, after every selection, re-solves for *all* chosen coefficients by least squares, so the residual is always orthogonal to the span of the atoms picked so far. That orthogonality is what distinguishes OMP from plain matching pursuit and prevents an atom from being selected twice. The two internal mechanics of this loop—which atom to add, and how to refit the residual—are described next.

2.2.1 Greedy atom selection

Starting from the full signal as the residual, $r_0 = y$, and an empty support set $\Lambda_0 = \varnothing$, each iteration measures how well every atom aligns with the part of the signal not yet explained. It correlates each atom with the current residual, $c_j = |\langle d_j, r_{i-1}\rangle|$, and greedily adds the single best-aligned atom to the support,

$$\lambda_i = \arg\max_j c_j, \qquad \Lambda_i = \Lambda_{i-1} \cup \{\lambda_i\}. \tag{1}$$

Because the residual is kept orthogonal to the already-selected atoms (§2.2.2), those atoms have zero correlation and cannot be chosen again, so each step commits to a genuinely new atom.

2.2.2 Residual update

Rather than keep the coefficients from earlier iterations, OMP recomputes the entire coefficient vector on the active set D_{Λ_i} by least squares at every step,

$$\hat{x} = \arg\min_x \|y - D_{\Lambda_i} x\|_2, \qquad r_i = y - D_{\Lambda_i} \hat{x}. \tag{2}$$

This orthogonal projection is the defining line of the algorithm: the residual r_i is orthogonal to every selected atom, so the next correlation measures only what the current approximation has not yet explained. The loop stops once the residual falls below the tolerance, $\|r_i\|_2 < \varepsilon$, or the sparsity cap $|\Lambda_i| = s$ is reached. The codes $\hat{x}$ gathered over all signals form the matrix X that the dictionary-update step of §2.3 then holds fixed while it refines the atoms.

This showcases how the orthogonal matching pursuit is able to congifure a sparse matrix for dictionary learning.

Algorithm 1 Orthogonal Matching Pursuit (sparse coding step)

Require: dictionary $D = [d_1, \dots, d_M]$, signal y, tolerance ε, sparsity cap s
Ensure: sparse code $\hat{x}$ with support Λ

Initialize: residual $r_0 \leftarrow y$, support $\Lambda_0 \leftarrow \varnothing$, $i \leftarrow 0$
repeat
  $i \leftarrow i + 1$
  Correlate: $c_j = |\langle d_j, r_{i-1} \rangle|$ for all atoms j ▷ §2.2.1
  Select: $\lambda_i = \arg\max_j c_j$
  Augment support: $\Lambda_i = \Lambda_{i-1} \cup \{\lambda_i\}$
  Project: $\hat{x} = \arg\min_x \|y - D_{\Lambda_i} x\|_2$ ▷ §2.2.2
  Update residual: $r_i = y - D_{\Lambda_i} \hat{x}$
until $\|r_i\|_2 < \varepsilon$ **or** $|\Lambda_i| = s$
return $\hat{x}$, Λ_i

2.3 K-SVD atom update through the restricted error matrix

With the codes X fixed by OMP, the dictionary-update step refines each atom in turn against the reconstruction DX. The matrix E it approximates is not the full residual but a *restricted* one. To update atom k, let Ω be the set of signals that use it, and form the error those signals leave for atom k to explain,

$$E = Y_\Omega - DX_\Omega + d_k \, x_{k,\Omega}^\top. \tag{3}$$

The final term is easy to misread as *adding* error; in fact it *un-subtracts* atom k. To see this, write the reconstruction DX as a sum over all atoms, where x^j denotes the j-th row of X,

$$DX = d_1 x^1 + d_2 x^2 + \cdots + d_k x^k + \cdots + d_K x^K. \tag{4}$$

The current residual therefore subtracts *every* atom's contribution, including atom k's,

$$Y - DX = Y - \underbrace{\left(d_1 x^1 + \cdots + d_K x^K\right)}_{\text{all atoms, including } k}. \tag{5}$$

Adding atom k's term back cancels it out of the sum, leaving the error that *every other* atom fails to explain,

$$(Y - DX) + d_k x^k = Y - \left(d_1 x^1 + \cdots + d_K x^K\right) + d_k x^k = Y - \sum_{j \neq k} d_j x^j. \tag{6}$$

This is exactly the gap a new d_k must fill: not the error left after atom k has already worked, but the error present when atom k is absent. Equation (3) is (6) restricted to Ω, and that restriction is what preserves the sparsity pattern: the update is computed only over signals that already use the atom, so the rank-one step never introduces the atom into signals whose codes were zero. This is the "restricted" in restricted error matrix, and it is the property that keeps the atom update and the sparse code consistent from one iteration to the next. Taking the SVD $E = U\Sigma V^\top$, the updated atom is the leading left singular vector $d_k = u_1$ and the updated coefficients are $x_{k,\Omega}^\top = \sigma_1 v_1^\top$, the best rank-one fit of the leftover error. Sweeping this update over all K atoms completes one dictionary-update step, after which the loop returns to sparse coding.

3 A Unified Framework for Tensor Dictionary Learning

3.1 The shared objective and the effective dictionary

Given training patches $\{\mathcal{Y}_m\}_{m=1}^M$, every method solves the same sparse-coding objective on the vectorized patches $y_m = \mathrm{vec}(\mathcal{Y}_m) \in \mathbb{R}^n$,

$$\min_{D,\{x_m\}} \sum_{m=1}^{M} \|y_m - Dx_m\|_2^2 \quad \text{s.t.} \quad \|x_m\|_0 \leq s, \quad \|d_k\|_2 = 1, \tag{7}$$

where $D = [d_1, \ldots, d_K]$ collects the atoms. The methods differ *only* in how the columns of the *effective dictionary* D are parameterized. Because Eq. (7) and its solver are shared, the accuracy comparison is apples-to-apples: sparse coding uses Orthogonal Matching Pursuit (OMP) [2] against D, and reconstruction is the linear map $x_m \mapsto Dx_m$, for all methods identically.

3.2 Where a decomposition can enter

A tensor decomposition can constrain the dictionary in two structurally different places, and this distinction organizes everything that follows.

- **Structure on each atom** (CP, TT, TR, BTD). Each atom is *itself* a low-parameter tensor. Since atoms are independent, the constraint is enforced atom-by-atom, by projecting the optimal unconstrained atom onto the decomposition's manifold inside the K-SVD update.
- **Structure on the whole dictionary** (Tucker). Every atom is built from a shared set of per-mode vectors, so the constraint couples all atoms; there is no single atom to project, and the update re-fits the shared mode factors instead.

3.3 Two algebraic bridges: Khatri–Rao and Kronecker

The link between a decomposition and the matrix form $Y \approx DX$ of DL runs through two products.

CP $\Rightarrow$ Khatri–Rao. A rank-one atom $a \circ b \circ c$ vectorizes to the Kronecker product of its factors. If atom k is built from column k of factor matrices A, B, C, then column k of the dictionary is $a_k \otimes b_k \otimes c_k$, so the entire dictionary is the column-wise Khatri–Rao product

$$D \;=\; A \odot B \odot C. \tag{8}$$

CP-DL therefore learns the small factors A, B, C; the atoms are the matched triples; and X is an ordinary sparse matrix selecting a few of the K atoms. Storage is $(n_1{+}n_2{+}n_3)K$ rather than $n_1 n_2 n_3\, K$.

Tucker $\Rightarrow$ Kronecker. A Tucker signal $\mathcal{Y} = \mathcal{G} \times_1 U_1 \times_2 U_2 \times_3 U_3$ vectorizes to

$$\mathrm{vec}(\mathcal{Y}) \;=\; (U_3 \otimes U_2 \otimes U_1)\ \mathrm{vec}(\mathcal{G}), \tag{9}$$

which is exactly $y = Dx$ with $D = U_3 \otimes U_2 \otimes U_1$ (the full Kronecker product) and $x = \mathrm{vec}(\mathcal{G})$ the (sparse) core. The Khatri–Rao is the column-matched Kronecker: CP keeps only matched triples (K atoms, a free sparse code), while Tucker keeps *all* $k_1 k_2 k_3$ combinations, with the core playing the role of the sparse code.

4 Methods

4.1 Tensor-decomposition contribution to Dictionary Learning

CP/TT/TR/BTD all represent the dictionary where the signal is described or transcribed through the tensor decomposition format of dictionaries, whcih tackles the problem regarding spacial contruction of data. With the Dicitonary Learning format following DX=Y, the following Tensor decomposition will take the following formation of D in dictionary learning as seen in Figure 1:

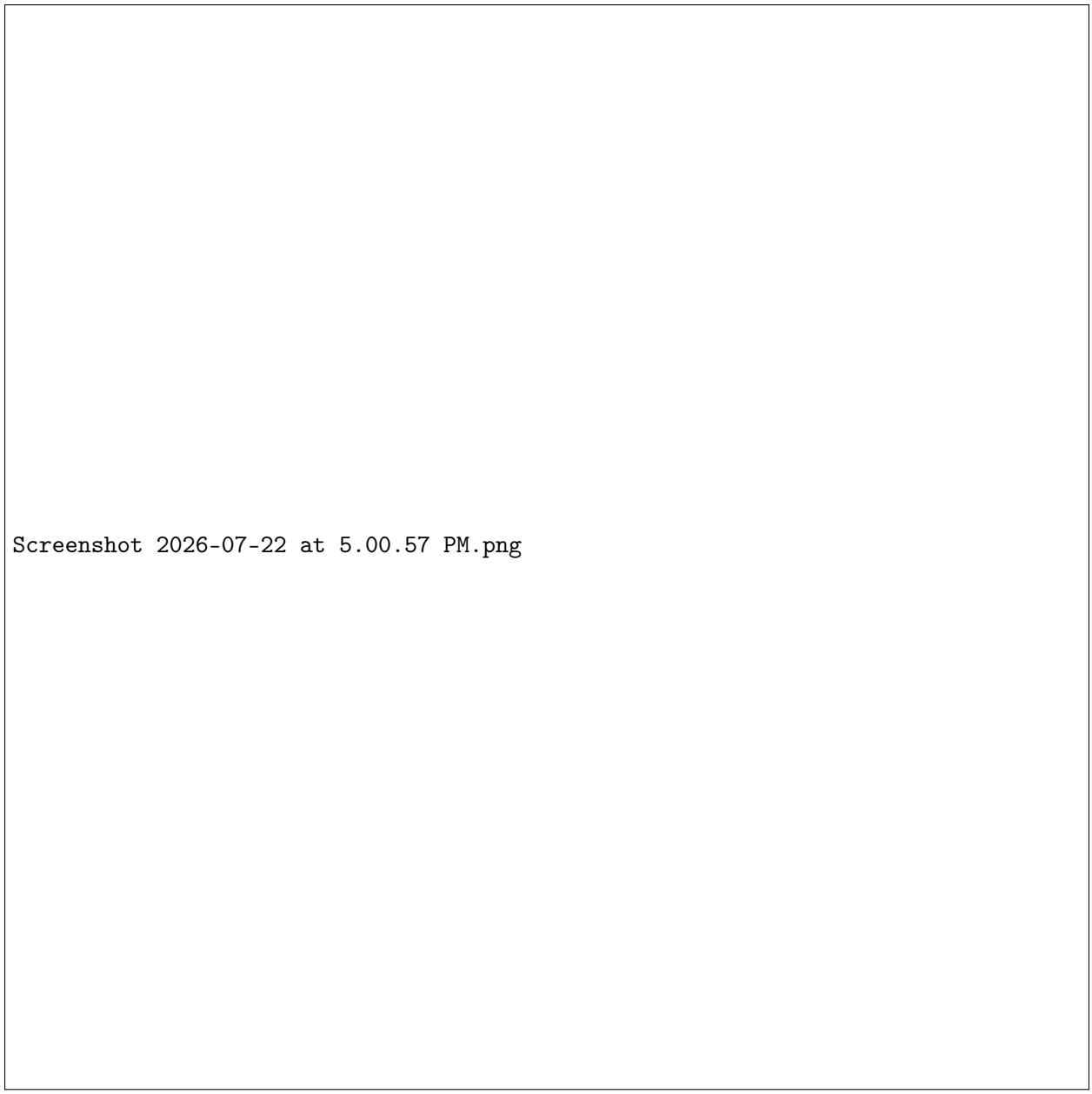


Figure 1: This showcases the tensor decomposition as dictionaries which are its advatages when there are three feature or modes

Figure 1 solely showcases when the dictionary is 3 dimensions but in larger datasets where there are more features the data that is continued in D explodes therefore through the tensor decomposition method the dictionaries can be represented in lower dimension at ehcih the data grows marginaly as compared to the normal dictioray block wich keeps the effeicny when computing Dictionary learning.

4.2 Tucker in the same harness

However Tucker decomposition doesnt represent the Dictionary. This is due to the fact that constraint is shared across atoms, it cannot be projected atom-by-atom. We keep the same outer

loop—OMP sparse coding against $D = U_3 \otimes U_2 \otimes U_1$, giving each sample a sparse core—then update the mode factors by separable alternating least squares. Holding the cores fixed, each U_n minimizes $\sum_m \|\mathcal{Y}_m - \mathcal{C}_m \times_1 U_1 \times_2 U_2 \times_3 U_3\|^2$; for mode n this is the linear solve $U_n = X_n G_n^\top (G_n G_n^\top + \lambda I)^{-1}$, where $G_m = \mathcal{C}_m \times_{p \neq n} U_p$. This "re-fit the shared factors" step is the Tucker analogue of the per-atom snap and is the canonical separable/Kronecker dictionary construction [8].

In this case we can clearly see that the core represents the sparse matrix whilst the 3 matrices represents the dictionary as seen in Figure 2.

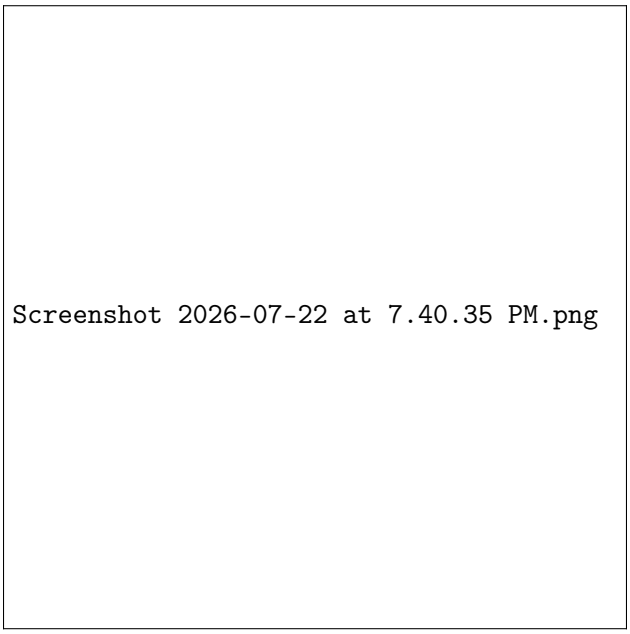


Figure 2: Enter Caption

4.3 Parameter accounting

Table 1 lists the free parameters stored per method for an order-N atom of shape $(n_1, \ldots, n_N)$ with $S = \sum_i n_i$. The dictionary parameter count is the storage budget we compare on; the sparse code x_m (or core) is per-sample and excluded from all methods equally.

Method	Dictionary form	Parameters
Vectorized	free atoms	$K \prod_i n_i$
CP (rank R)	$D = A \odot B \odot C$ (Khatri–Rao)	KRS
Tucker	$D = U_N \otimes \cdots \otimes U_1$ (Kronecker)	$\sum_i n_i k_i$
TT (rank r, N=3)	chain of cores	$K\big[r(n_1{+}n_3) + r^2 n_2\big]$
TR (rank r)	ring of cores	Kr^2S
BTD (L, b blocks)	sum of Tucker blocks	$Kb\,(L^N + LS)$

Table 1: Dictionary parameter cost per method. K is the number of atoms; for Tucker the atom count is $\prod_i k_i$ and the stored parameters are only the mode factors.

5 Experimental Setup

We use three datasets. **Synthetic mixed-structure** patches of shape $6 \times 6 \times 6$ are built from ground-truth atoms that are 40% separable (rank-one), 40% low-TT-rank, and 20% dense, combined with true sparsity 3 and Gaussian noise, so no structure is trivially favoured. **Real RGB** patches of shape $8 \times 8 \times 3$ are extracted from a standard photograph and mean-removed/normalized. **Hyperspectral (HSI)** patches of shape $6 \times 6 \times 16$ come from a synthetic cube generated by mixing a few smooth spectral signatures with smooth spatial abundance maps, giving a genuinely low-rank, fat-third-mode signal. Dictionaries are learned on noisy training patches and evaluated by denoising PSNR against clean held-out test patches; results are averaged over random seeds. Decomposition primitives use TensorLy [10].

6 Efficiency Ranking System

"Efficiency" of a decomposition for DL has multi-objective goals in terms of its accuracy, parameter thrift, training speed, and robustness. Therefore we rank in three layers from objective to opinionated.

Layer 1: Pareto dominance (weight-free). On the two core axes, test accuracy and parameter-efficiency, a method is *dominated* if another is at least as good on both and strictly better on one; no non-negative weighting can rescue a dominated method. The non-dominated methods form the rate–distortion frontier.



Figure 3: Enter Caption

Layer 2: an interpretable scalar.

Their accuracy measure is :

$$\mathbf{E} \;=\; \frac{\mathrm{PSNR}}{\log_{10}(\mathrm{params})},$$

where PSNR is the peak signal-to-noise ratio (in decibels).

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(x_i - \hat{x}_i)^2 \tag{10}$$

$$\mathrm{PSNR} = 10\log_{10}\left(\frac{\mathrm{MAX}^2}{\mathrm{MSE}}\right) \tag{11}$$

This formula presumes a reconstruction/compression task — you need a signal being approximated and an error to measure against it.

The denominator, $\log_{10}(\mathrm{params})$, counts *decades* of parameters: going from 10^4 to 10^5 parameters is one decade. So E reads as *decibels of fidelity retained per order of magnitude of parameters spent* — a compactness-adjusted accuracy score.

Its selling points: it requires neither storing model weights nor normalizing against a reference/competitor method, since both PSNR and $\log_{10}(\text{params})$ are computable from the model alone.

Layer 3: profile-weighted tiers. Each axis is Min–Max normalized to $[0, 1]$ (higher is better) and combined by a weighted sum under a named profile (Table 2) where in different cases there are different use cases for **Balanced**, **Quality**, **Compression** and **Real time**; therefore, each tensor decompsition structure is graded based on grade factors which are given through the following score S ≥ 0.80, A ≥ 0.65, B ≥ 0.45, C ≥ 0.25, else D.

Given each persentages in (Table 2) for Balanced, Quality, Compression, Real time there will be a score given to each tensor decomposition in terms of Dictionary Learning ranging from A to D

Profile	Accuracy	Param. eff.	Speed	Robustness
Balanced	0.40	0.30	0.15	0.15
Quality (offline)	0.60	0.20	0.10	0.10
Compression (edge)	0.25	0.55	0.10	0.10
Realtime	0.30	0.20	0.40	0.10

Table 2: Use-case weight profiles for the composite efficiency score.

7 Results

7.1 Synthetic mixed-structure data

Table 3 and Figure 4 give the efficiency frontier at 12 dB. The unstructured baseline attains the highest accuracy (41.5 dB) but at the largest dictionary (25,920 parameters) which is warranted as there are no restrictins on the vectorized format as shown through the immense number of parameters. CP is the most compact per atom (12× smaller) which in terms of dictionary learning represents the tensor in greatest depth, but least accurate at fixed atom count. TT and TR occupy the middle.Tucker (here learned by a separate per-mode procedure) is astonishingly small (126 parameters, 206× compression) at the cost of accuracy.

Model	Atoms	Params	Compression	Fit (s)	PSNR (dB)
Vectorized	120	25,920	1.0×	0.85	41.49
CP	120	2,160	12.0×	3.50	33.14
Tucker	343	126	205.7×	1.42	27.01
TT	120	5,760	4.5×	1.13	38.29
TR	120	8,640	3.0×	1.24	38.95

Table 3: Synthetic mixed-structure patches ($6 \times 6 \times 6$, 12 dB, K=120, encode sparsity 6, 5 seeds).

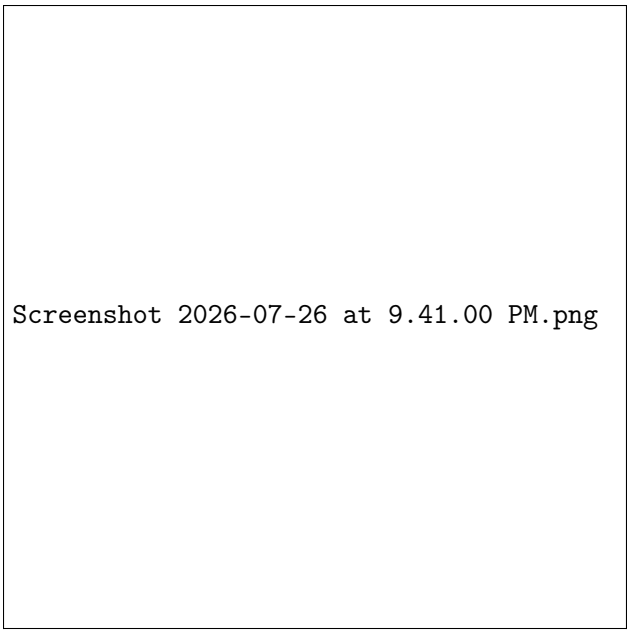


Figure 4: Efficiency frontier on synthetic data: test PSNR versus dictionary parameters (log scale). Up-and-left is better.

All three of CP, TT, and TR start at the identical point (33.1 dB) at rank 1. That is due to the fact that at rank 1 all three decompositions are the same with it expanding into an outerproduct functions $a \circ b \circ c$. CP lags and wobbles as the CP sits 2 dB below TT/TR at rank 2 (36.6 vs 38.3–38.8) and needs rank 3 just to reach 39 dB. The error bars with CP shows the variance across random seeds, and this is the empirical fingerprint of exactly what we discussed about CP fitting being not as accurate as there is no SVD initialization for CP rather utilizing the alternating least squares method, so different runs land in different places. TT and TR's tight bars reflect their stable, quasi-optimal fitting. The way Tucker atoms are structured in this experiment imposes a form of per-mode separability that the data fundamentally doesn't have, so the error floor is structural. The CP/TT/TR curves nearly collapse onto each other in the mid-range. In the left panel CP looked clearly worse, but CP's rank is also cheaper (R·S parameters, versus TT's r^2-flavored cost). So when you charge each method its true price, CP's points slide leftward relative to TT's, and its lower PSNR is compensated by lower cost: at 5×10^3 parameters, CP's rank-3 point (39.1 dB) sits essentially on top of TT's rank-2 trajectory. Per parameter, the three are far more similar than the left panel suggested.

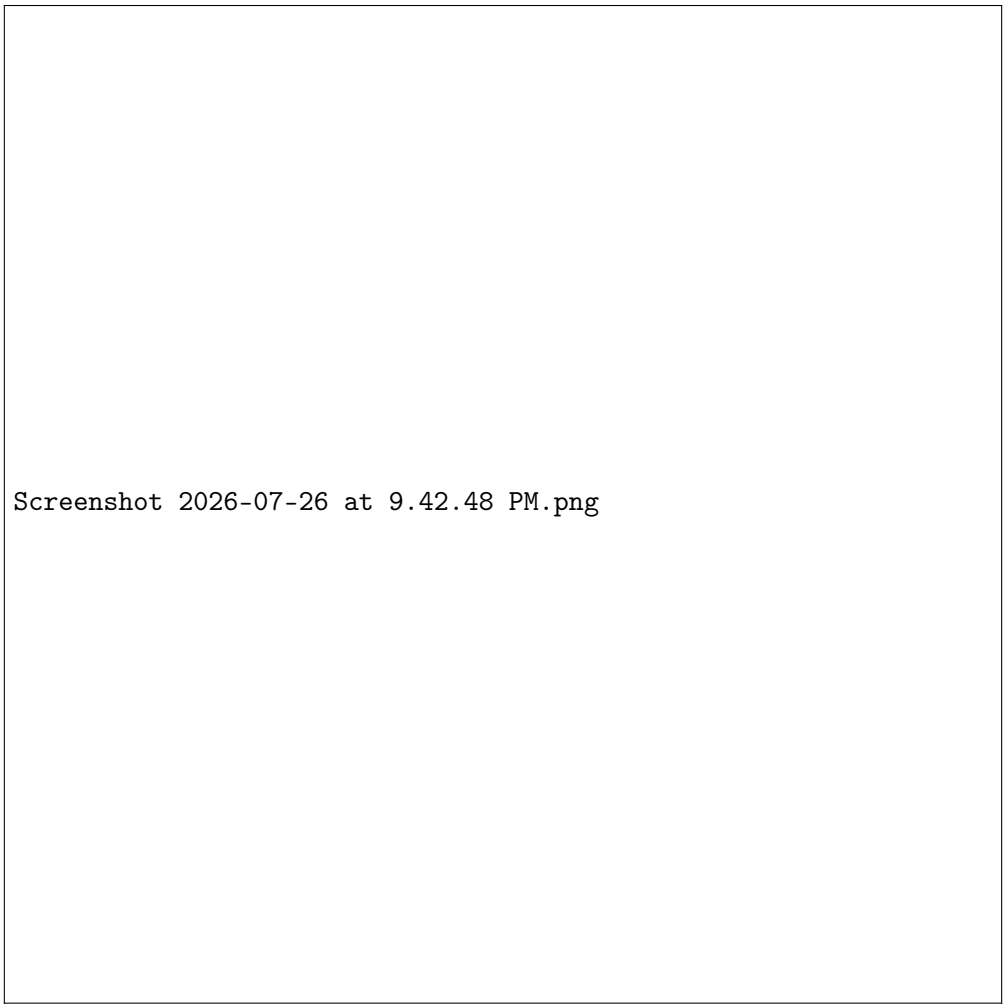


Figure 5: Rank sensitivity on synthetic data. Left: PSNR versus each method's rank knob. Right: PSNR versus dictionary parameters under the rank sweep. TT overtakes the baseline at rank 4; Tucker is capped by separability.

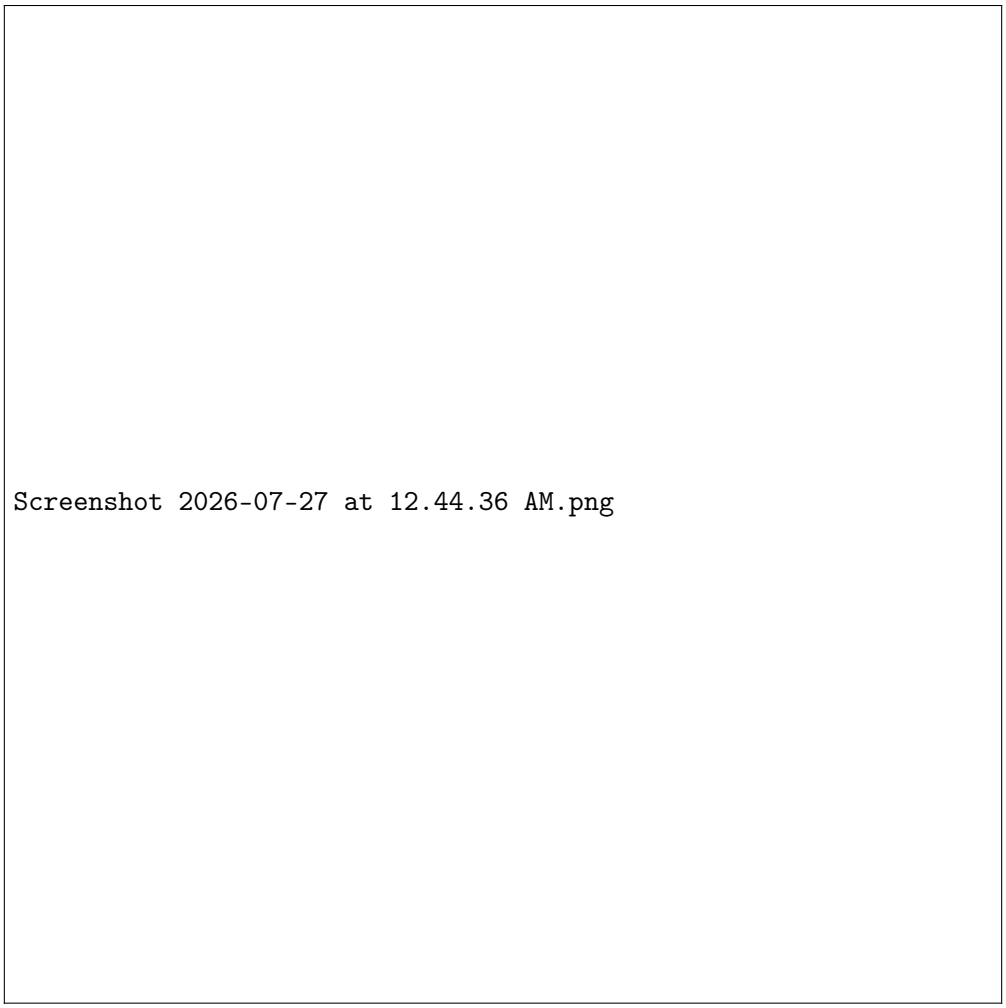


Figure 6: Initialization sensitivity across random seeds. Taller boxes are less robust; the unstructured baseline varies most, the constrained models least.

7.2 Real RGB patches: the regime flips

On real image patches (Table 4) the conclusion inverts. Every structured method except the initial Tucker *beats* the unstructured baseline while using far fewer parameters: CP reaches 31.65 dB at 10× compression versus the baseline's 30.81 dB. On real, intrinsically low-rank patches the structural constraint is no longer a handicap but a regularizer that prevents the over-parameterized free dictionary from fitting noise (Figure 7).

Model	Atoms	Params	Compression	Fit (s)	PSNR (dB)
Vectorized	100	19,200	1.0×	0.82	30.81
CP	100	1,900	10.1×	2.54	**31.65**
Tucker†	243	153	125.5×	1.41	30.74
TT	100	5,400	3.6×	1.06	31.27
TR	100	7,600	2.5×	1.18	31.27
BTD	100	4,600	4.2×	3.00	31.37

Table 4: Real RGB patches ($8 \times 8 \times 3$, 12 dB, encode sparsity 6). †Initial Tucker (separate optimizer); corrected in Section 7.3.

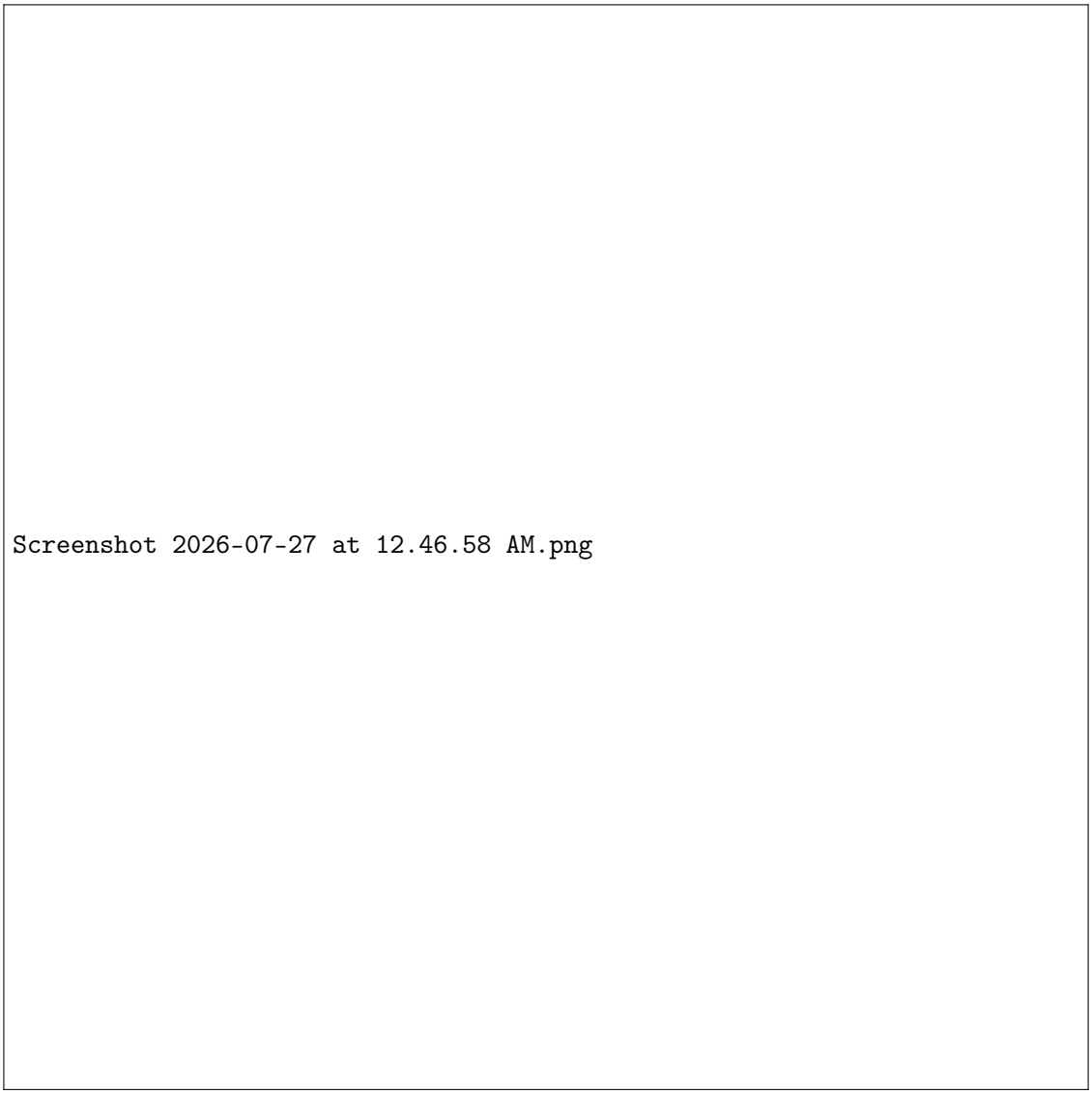


Figure 7: Efficiency frontier on real RGB patches. Structured dictionaries move above the vectorized baseline; the initial Tucker matches it at 125× fewer parameters.

CP instability. Figure 8 isolates the classic CP degeneracy. We measure a degeneracy ratio (sum of component magnitudes divided by the norm of their sum): rank-one atoms are always

non-degenerate (= 1.0), and the ratio grows with rank, with random initialization producing markedly more degenerate, partially cancelling components than SVD warm-start (2.35 vs 1.62 at rank 3). Notably, reconstruction accuracy can look fine while the underlying factorization is internally degenerate—a warning if the factors themselves are to be used downstream.

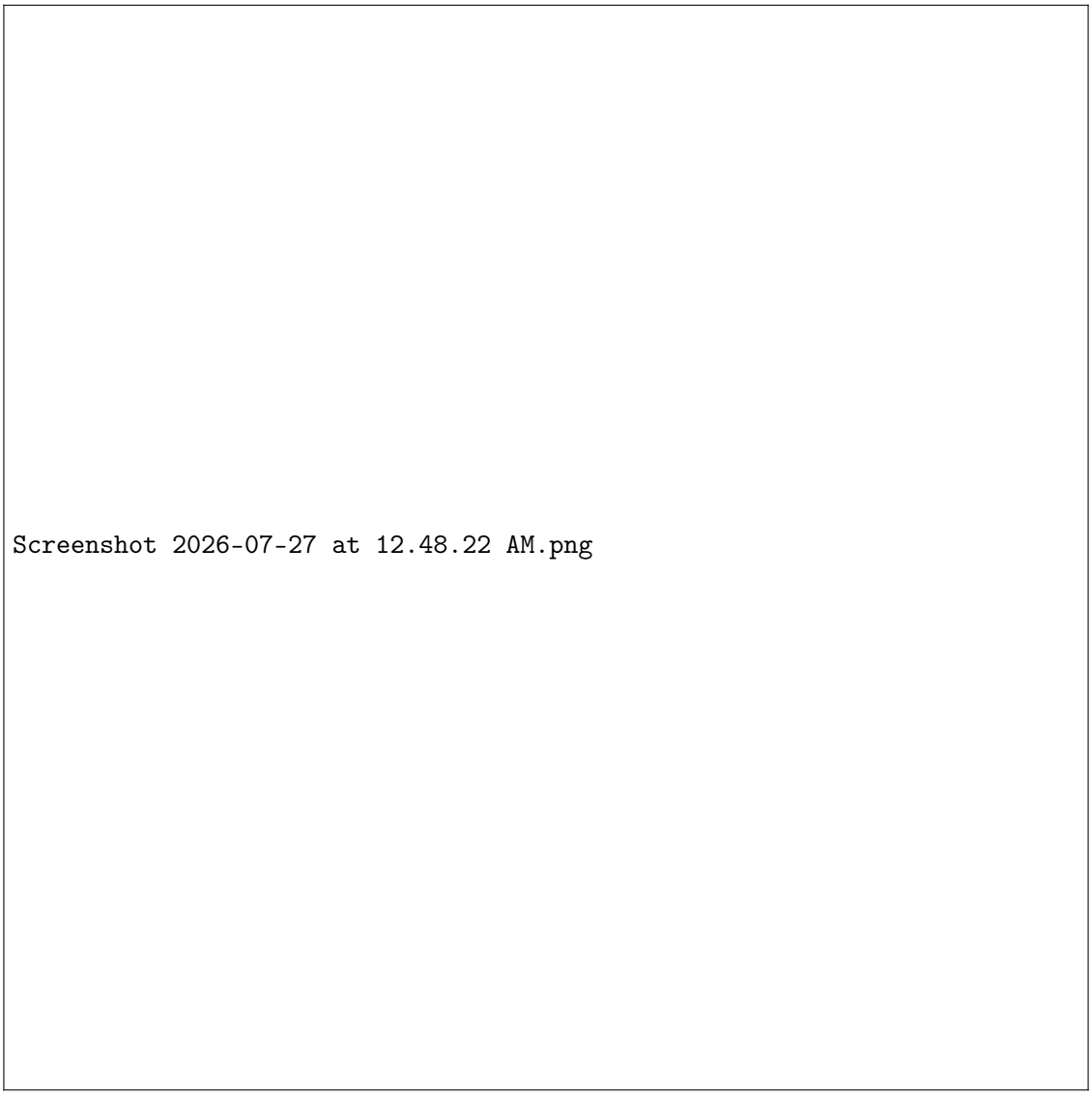


Figure 8: CP instability on real patches. Left: accuracy and seed-spread versus rank for SVD vs random initialization. Right: the degeneracy diagnostic climbs with rank and is worse under random init.

BTD interpolates CP and Tucker. Figure 9 sweeps BTD's block rank L and block count. The L=1, one-block configuration lands exactly on CP rank-one, confirming the endpoint identity, and the family traces the continuum toward Tucker. In this noisy regime the most constrained point is best; extra capacity fits noise.

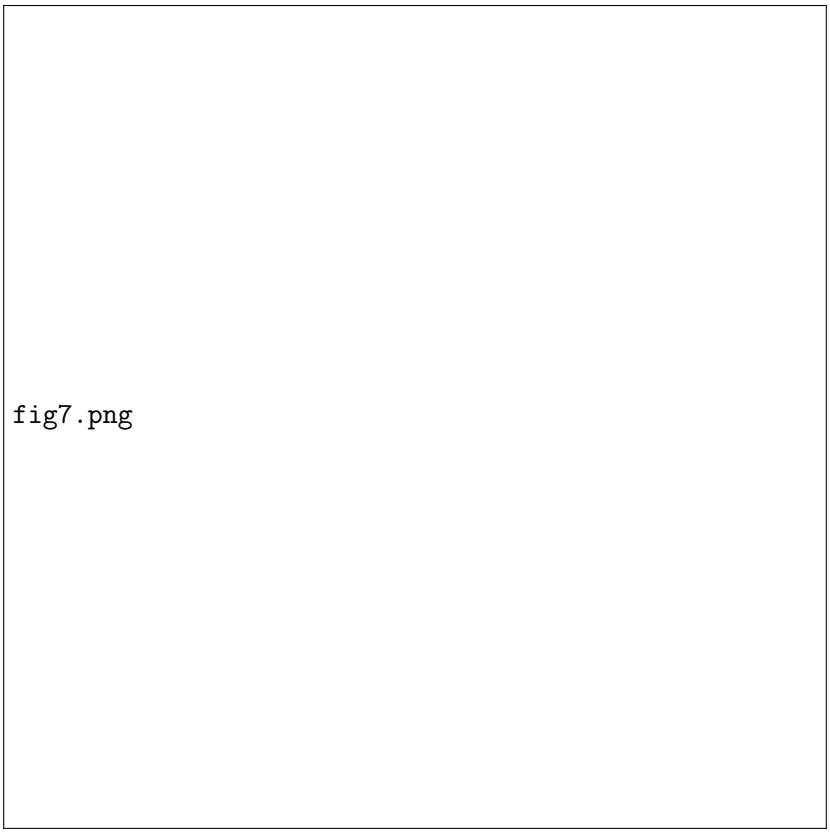


Figure 9: BTD traces the CP $\leftrightarrow$ Tucker continuum as block rank L and the number of blocks vary, overlaid on CP (rank 1–3) and Tucker reference points.

7.3 A fairness correction to Tucker

The initial Tucker of Table 4 was learned by a procedure different from the shared harness. Rebuilding Tucker into the harness (Section 4: OMP coding plus separable ALS) raises it from 30.74 to 32.16 dB at the identical 153 parameters, and *halves* its seed spread ($0.51 \rightarrow 0.09$). The earlier weakness was largely an artifact of the optimizer, not of the Tucker structure—a concrete instance of an implementation confound dominating a structural difference.

7.4 Efficiency ranking on real patches

With the corrected Tucker, the ranking system (Table 5, Figure 10) is decisive: Tucker is the sole Pareto-optimal method—highest accuracy *and* fewest parameters—so it dominates every profile (tier S), and the previously winning CP drops to tier C. The objective layer alone reduces the field: on the accuracy–parameter plane, every method other than Tucker is dominated.

Model	PSNR	Params	E	Pareto	Tier	Balanced
Tucker	32.16	153	14.72	**yes**	S	0.892
CP	31.71	1,900	9.67	no	C	0.429
TT	31.28	5,400	8.38	no	C	0.389
BTD	31.38	4,600	8.57	no	C	0.385
TR	31.31	7,600	8.07	no	C	0.306
Vectorized	30.80	19,200	7.19	no	D	0.230

Table 5: Efficiency ranking on real RGB patches (corrected Tucker, 5 seeds). $E = \text{PSNR}/\log_{10}(\text{params})$; "Balanced" is the composite score.

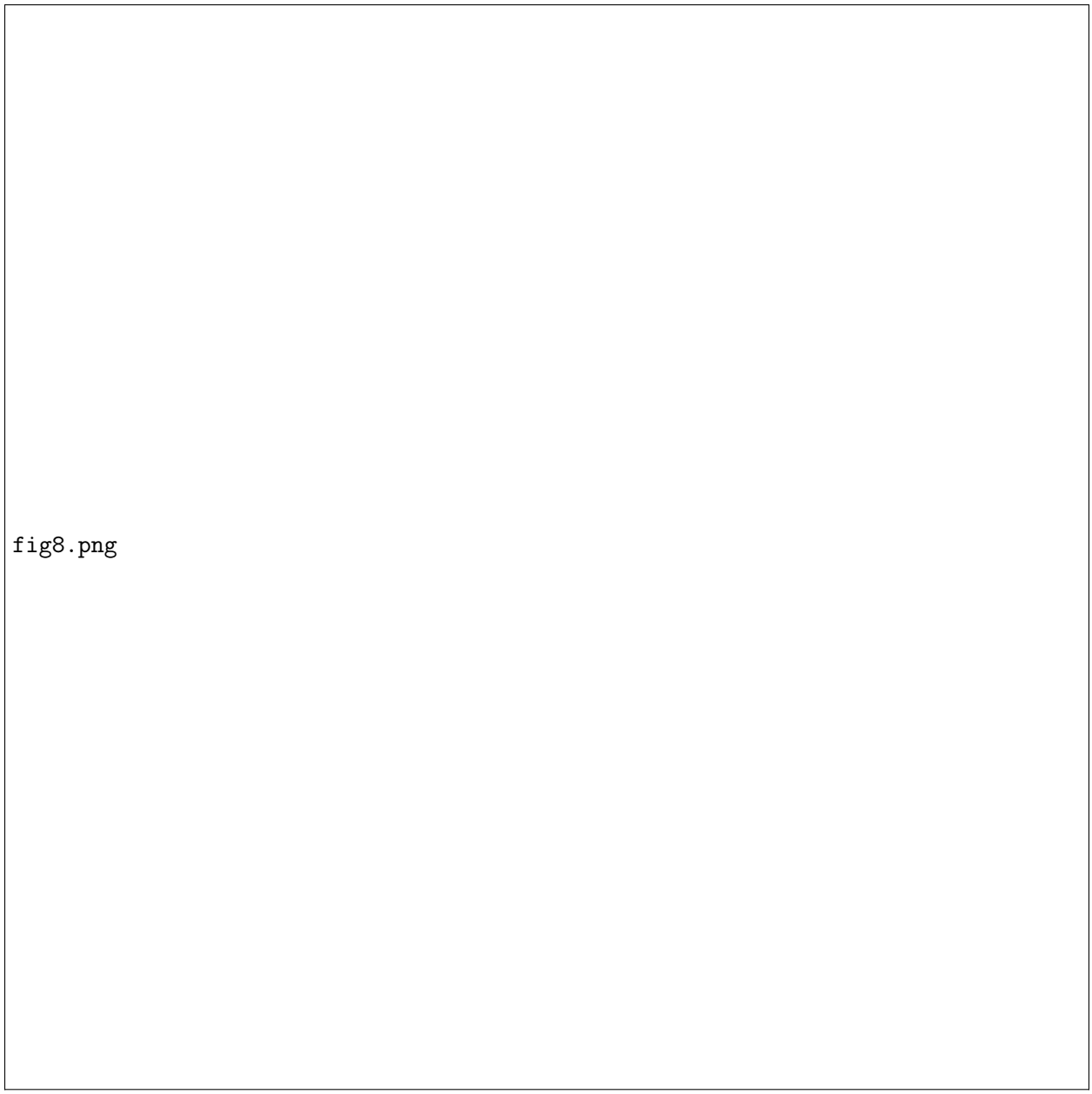


Figure 10: Ranking system. Left: objective Pareto layer (stars are non-dominated); only Tucker is on the frontier. Right: profile-weighted composite scores.

7.5 Matched-budget rate–distortion on hyperspectral data

The comparisons above use hand-chosen ranks and atom counts. To remove that confound we fix a parameter budget and let each method auto-select its rank/atom count (validation-best) to spend it, sweeping the budget to trace a rate–distortion curve, and repeating across noise levels on the HSI cube (Figure 11, Table 6). Two findings stand out. First, giving Tensor-Train a large third mode did *not* hand it the win: a single fat mode does not favour TT for DL, whose advantage is high tensor *order*, not one wide mode. Second, the ranking does not flip between heavy (6 dB) and light (18 dB) noise; only the absolute PSNR shifts. On this intrinsically low-rank cube, accuracy falls as the budget grows (clearest at 6 dB): the most regularized model wins, and the baseline

leads at 5,000 parameters partly because its atoms cost 576 each and it is forced into a few, heavily regularized atoms. Tucker sits at the far left of the curve (it cannot spend a 5,000-parameter budget without exceeding a reasonable atom cap), so "best at 5,000 parameters" understates its per-parameter efficiency.

Model (at ≈5,000 params)	6 dB PSNR	18 dB PSNR
Vectorized	**32.27**	**41.80**
CP	29.91	41.08
TT	29.24	40.58
TR	29.16	40.54
BTD	28.49	40.21
Tucker*	26.90	38.82

Table 6: Matched-budget ranking on the HSI cube ($6 \times 6 \times 16$) at the ≈5,000-parameter slice. *Tucker operates at 304 parameters (it cannot reach the budget); see text.

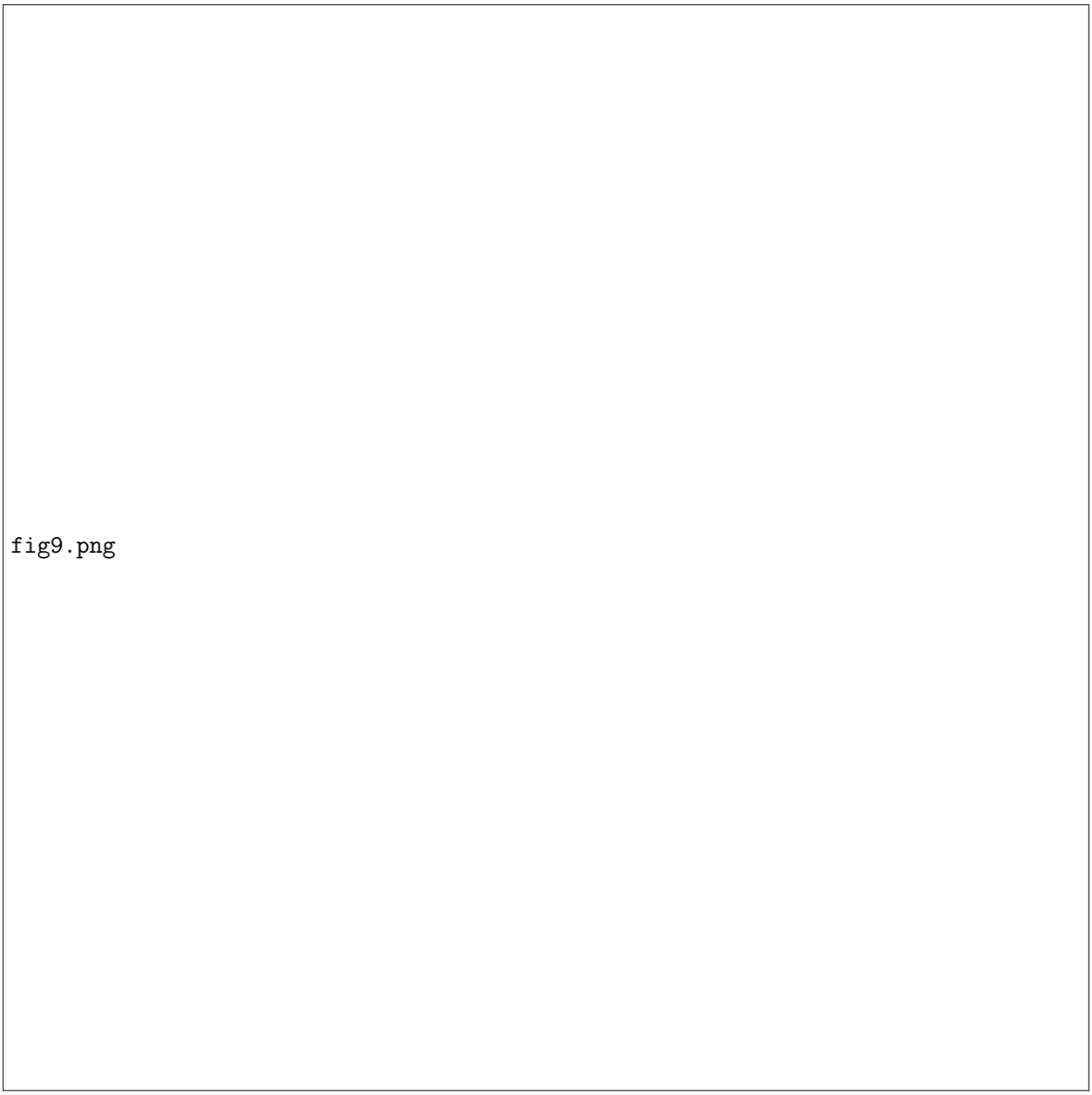


Figure 11: Matched-budget rate–distortion on the HSI cube at 6 dB (heavy) and 18 dB (light) noise. Each point auto-selects its rank/atoms to hit the budget; the dotted line marks the 5,000-parameter slice. The ordering is stable across noise, and a fat third mode does not favour TT/TR.

8 Discussion

Three themes emerge. **(i) Regime dependence.** There is no absolute best decomposition: the unstructured baseline wins on rich clean synthetic data; structured dictionaries win on noisy real patches where their constraint regularizes; and at matched budget on low-rank HSI, few full atoms or a tiny Tucker are most efficient. Which decomposition to use is a function of signal order, noise, and intrinsic rank, not a fixed ranking. **(ii) Confounds can dominate structure.** The Tucker rebuild (Section 7.3) moved it from last to first at identical size purely by fixing the optimizer,

and the matched-budget study revised the earlier "structured wins" story by revealing that some of that advantage was really the freedom to buy many cheap atoms. **(iii) Matched-parameter is not matched-atom.** At equal storage, the baseline is forced into few heavily regularized atoms while structured methods spend the same budget on many atoms; which budget is "fair" depends on whether one cares about storage or per-atom quality.

9 Limitations

We are explicit about scope. The CP/TT/TR/BTD methods are *projection variants* inside a shared K-SVD loop, not the canonical published algorithms (e.g. jointly optimized factors, PALM-based TT-DL, or SeDiL); the numbers rank these implementations, and the labels should not be read as those methods' definitive performance. There is no single "correct" algorithm per decomposition—each admits many optimizers—so a fully faithful leaderboard would report several implementations per structure and show the within-structure spread. Our datasets are modest (hundreds of patches, single images, few noise levels), the error bars capture only seed variance, and the HSI cube is synthetic. The matched-budget study caps atom counts for tractability, which bounds Tucker artificially. Finally, the harness materializes the full effective dictionary and runs identical OMP for fairness, which discards the fast structured transforms that are a genuine computational advantage of Kronecker/TT dictionaries; the reported "speed" axis therefore understates the tensor methods. These are the natural next steps rather than flaws to hide.

10 Conclusion

We presented a single harness in which five tensor-structured dictionaries and an unstructured baseline differ only by how each atom is parameterized, connected them to dictionary learning through the Khatri–Rao (CP) and Kronecker (Tucker) forms, and built a three-layer efficiency ranking system. Across synthetic, real, and hyperspectral data the best decomposition is regime-dependent; implementation confounds can outweigh structural differences; and a matched parameter budget both tightens and, in places, reverses naive conclusions. The framework is designed so that new decompositions are one projector and one parameter rule away, and so that the ranking can be re-derived on any dataset.

References

[1] M. Aharon, M. Elad, and A. Bruckstein. K-SVD: An algorithm for designing overcomplete dictionaries for sparse representation. *IEEE Transactions on Signal Processing*, 2006.

[2] Y. C. Pati, R. Rezaiifar, and P. S. Krishnaprasad. Orthogonal matching pursuit: recursive function approximation with applications to wavelet decomposition. *Asilomar Conference on Signals, Systems and Computers*, 1993.

[3] T. G. Kolda and B. W. Bader. Tensor decompositions and applications. *SIAM Review*, 2009.

[4] L. R. Tucker. Some mathematical notes on three-mode factor analysis. *Psychometrika*, 1966.

[5] I. V. Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 2011.

[6] Q. Zhao, G. Zhou, S. Xie, L. Zhang, and A. Cichocki. Tensor ring decomposition. *arXiv:1606.05535*, 2016.

[7] L. De Lathauwer. Decompositions of a higher-order tensor in block terms—Part II: Definitions and uniqueness. *SIAM Journal on Matrix Analysis and Applications*, 2008.

[8] S. Hawe, M. Seibert, and M. Kleinsteuber. Separable dictionary learning. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2013.

[9] C. F. Caiafa and A. Cichocki. Computing sparse representations of multidimensional signals using Kronecker bases. *Neural Computation*, 2013.

[10] J. Kossaifi, Y. Panagakis, A. Anandkumar, and M. Pantic. TensorLy: Tensor learning in Python. *Journal of Machine Learning Research*, 2019.